Enso: Learned Per-Request Routing to Frontier Intelligence, and Enso Ultra's Training-Free Fan-Out Synthesis

Hanzo AI Research

Hanzo AI

Hanzo Research

Enso: Learned Per-Request Routing to Frontier Intelligence, and Enso Ultra's Training-Free Fan-Out Synthesis

Zach Kelling*
Hanzo Industries Inc (Techstars '17)
Los Angeles, California
`https://hanzo.ai`

Hanzo Research

Abstract

The strongest answer to a given request rarely comes from the same model twice. On hard software engineering a frontier reasoning model wins; on long-context synthesis a million-token model wins; on cheap high-volume traffic the marginal gain of the most expensive model is zero. A single monolithic model is therefore a bet that one weight set dominates every request in the stream, which the published per-benchmark spread between frontier models refutes. We describe **Enso**, Hanzo's proprietary closed model family that turns this spread into a product. `enso` is a learned router: it featurizes each request, scores every arm in a pool of frontier models by a bilinear utility $x^\top W p$, and returns the single SLO-feasible arm that maximizes quality $-\,\lambda\,\text{cost} - \mu\,\text{latency}$, adapting to each user online. `enso-ultra` reuses the same policy to select the top-N arms, dispatches them in parallel, and fuses their answers with a training-free synthesizer. The routing policy is the one specified in the Hanzo Router paper [13]; this paper is about the family it powers, the serving substrate that presents it as one `/v1` API, and how it compares to frontier baselines (Opus 4.8, GPT-5.5, GPT-5.6, Gemini 3.1 Pro, Fable 5) and to a prior published learned orchestrator from the recent literature [16]. Our design choice is deliberate and stated plainly: unlike the prior published router (Ultra), whose orchestrator is trained with reinforcement learning, Enso is *training-free at the orchestration layer and learned only at the routing layer*, so a new frontier arm is onboarded by adding a row, not by retraining a policy over the pool. We report a measured evaluation from the `enso-bench` harness—every arm and every SKU executed under one gateway on the same items, with a per-request cost column the standard router table lacks. On GPQA Diamond `enso` scores 92.9% at \$11.1 per 1,000 tasks, matching the best single arm (`gpt-5.6-sol`, 91.9%) and beating the premium reasoning arm (Opus 4.8, 86.4% at \$55.4/1k) at roughly a fifth of its metered cost, while `enso-ultra`'s verify-then-select fan-out is *both* more accurate and cheaper than blind synthesis. We are equally explicit where routing does *not* win—it leads neither every benchmark nor Humanity's Last Exam—and typeset any unmeasured cell as *run pending* rather than invent a number.

1 Introduction

A frontier language model is a fixed bet. Its weights encode one balance of code, reasoning, math, chart understanding, long-context recall, and instruction following, and that balance is frozen at training time. But the request stream a production assistant sees is not fixed: one request is a graduate-level physics question, the next is a repository-scale refactor, the next is a chart to read, the next is a one-line reformat. The public per-benchmark tables make the consequence concrete. On the same seven benchmarks, the frontier models trade the lead back

and forth—one leads on SWE-Bench Pro, another on LiveCodeBench Pro, another on Humanity's Last Exam—and no single model holds every column. A monolithic deployment picks one of these models and eats its losses on every column it does not lead. That is the structural inefficiency Enso removes.

The move is to stop shipping a model and start shipping a *policy over models*. Enso is a closed family with two SKUs behind one API. `enso` routes each request to the single frontier arm most likely to serve it best under a per-request service-level objective. The pool we measure here is the set our gateway can actually serve—GPT-5.6 (sol), Claude Opus 4.8, GPT-5.5, Claude 5 Sonnet, DeepSeek-V4-Pro, and Claude Fable 5, with GPT-4o as a cheap baseline single—and it spans the same capability axes a router needs: frontier reasoning, strong generalists, a dedicated coder, and a long-context arm. Gemini 3.1 Pro has no endpoint on this gateway, so it appears only as a provider-reported reference column, never as a routable arm. `enso-ultra` spends more to buy the ceiling: it fans the request out to the top-N arms in parallel and synthesizes one answer from their candidates. Both SKUs are a single `/v1` endpoint; the caller sends messages and gets an answer, and never has to know—or maintain keys for—the providers underneath.

This framing has two consequences we make central. First, **routing beats monoliths whenever the pool's per-request argmax beats any fixed member**, and the published spread shows that condition holds. A learned router that identifies the right arm per request inherits the pool's upper envelope, not any single model's average. Second, **a router makes cost a first-class, transparent axis**. A monolith charges its one price on every token regardless of whether the request needed it; a router that optimizes quality $-\lambda$ cost $-\mu$ latency sends cheap requests to cheap arms and prices each answer at the arm that actually served it. We therefore report a cost-per-benchmark column alongside accuracy—the number a buyer of intelligence actually pays—which comparison tables in this line of work typically omit.

We are explicit about lineage. The routing mechanism, the SLO objective, the bilinear utility, and the online per-user adaptation are specified and analyzed in the Hanzo Router paper [13]; we cite that work for the policy internals rather than restating them, and focus here on (i) the Enso *family* and the serving substrate that presents it as one product (§3), (ii) the Enso Ultra fan-out/synthesis SKU and why its orchestration is deliberately training-free (§4), and (iii) a measured evaluation against the same benchmarks a learned-orchestrator baseline reports (§5).

2 Related Work

Learned orchestration: prior router, Trinity, Conductor. The closest system to Enso is the prior published router [16], a learned orchestrator over a pool of frontier models—Gemini 3.1 Pro, Claude Opus 4.8, and GPT-5.5—shipped in two SKUs, a base tier and an Ultra tier, exactly the two-tier shape Enso takes. The base tier is trained with supervised fine-tuning plus separable CMA-ES [14], and its Ultra tier adds GRPO reinforcement learning [18] at the orchestration layer; both build on the ICLR 2026 foundations Trinity [17] (evolutionary composition of experts) and Conductor [15] (RL-orchestrated ensembles). Enso shares that router's thesis—that the per-request argmax over a frontier pool beats any fixed member—and its two-SKU packaging, and we treat its published results as the reference point our evaluation mirrors (§5). Enso differs in where the learning sits: the prior router's Ultra tier learns an orchestrator with RL over the pool, whereas Enso learns only the routing policy and keeps orchestration training-free (§4), so onboarding a new arm is a data change, not a retrain.

Routing and cascading. Enso's single-arm SKU is a router in the line of RouteLLM [10], which learns a strong/weak binary router from preference data; FrugalGPT [1], which cascades cheap-to-expensive with a learned stopping scorer; Hybrid LLM [2], which routes on a

quality-aware threshold; AutoMix [9], which escalates via self-verification; and Tabi [21], a multi-level inference system. Enso generalizes their binary or cascade decision to a constrained continuous objective, quality $-\lambda$ cost $-\mu$ latency, over a pool of many frontier arms, and adds per-user online adaptation [8] that cascade work generally omits. The full treatment is in the router paper [13].

Ensemble synthesis. Enso Ultra's fan-out/synthesis is grounded in multi-agent and ensembling work: Mixture-of-Agents [19] layers proposer and aggregator models; LLM-Blender [6] ranks and generatively fuses candidates from a model bank; and self-consistency [20] shows that aggregating multiple independent samples improves reasoning. Ultra applies these fusion ideas across *heterogeneous frontier arms* rather than samples from one model, and keeps the fuser training-free.

3 Enso Architecture

Enso is two layers with one seam between them: a *routing layer* that decides which arm(s) serve a request, and a *serving layer* that presents the family as one metered product. Figure 1 shows both SKUs.

3.1 The routing layer: policy as data

The router is the mechanism and policy specified in the Hanzo Router paper [13]; we summarize only what the family needs and cite that work for the rest. Each arm is one row in a cross-modal registry—a `(model, level, modality)` profile carrying an eval-measured quality vector over tasks plus normalized latency, cost, VRAM, and context reach. A request is turned into a feature vector x by a hashing-trick featurizer (task one-hot, hashed n-grams, log-length, media flag, bias) in sub-microsecond time, and every arm profile p is scored by the learnable bilinear utility

$$u(x,p) = x^\top W p, \qquad W \in \mathbb{R}^{D\times K}. \tag{1}$$

The selector returns the SLO-feasible arg max of the constrained objective

$$\arg \max_{p\,:\,\text{feasible}(p)} \; u(x,p) - \lambda\,\widehat{c}(p) - \mu\,\widehat{\ell}(p), \tag{2}$$

where the hard ceilings (`max_latency_ms`, `max_cost`, `min_quality`) gate the candidate set and the soft weights λ, μ trade cost and latency against quality. Offline, W is a closed-form ridge fit [4]; online, each user carries a LinUCB bandit [8] whose parameter θ_u starts at W and drifts toward that user's taste as outcomes arrive through a single `observe`(u, x, p, r) call. The essential property for the family is that *the policy is a function of data*: adding a frontier arm is adding a profile row, and its ranking then falls out of the same W without a code change. A two-tier safety guard gates every request before the selector runs, cheaply on the hot path and escalating only ambiguous prompts.

3.2 The serving layer: family-as-data

The serving substrate (the `zen-svc` engine behind the Hanzo AI gateway [3]) turns the routing decision into one product. Three properties matter.

One identity. Whichever arm serves a request, the response is presented as Enso. The serving layer performs identity injection and upstream-hiding: the caller sees a single model family and its provenance, not the underlying provider, and never manages third-party keys. This is what lets the pool change—an arm added, retired, or swapped for a better version—without any client change.

One meter. Every routed request is metered on one plane, so cost is attributed per request to the arm that actually served it. This is the substrate that makes the cost-per-benchmark column of §5 a real, per-request number rather than a blended list price, and it is what lets a deployment set λ to move spend without touching application code.

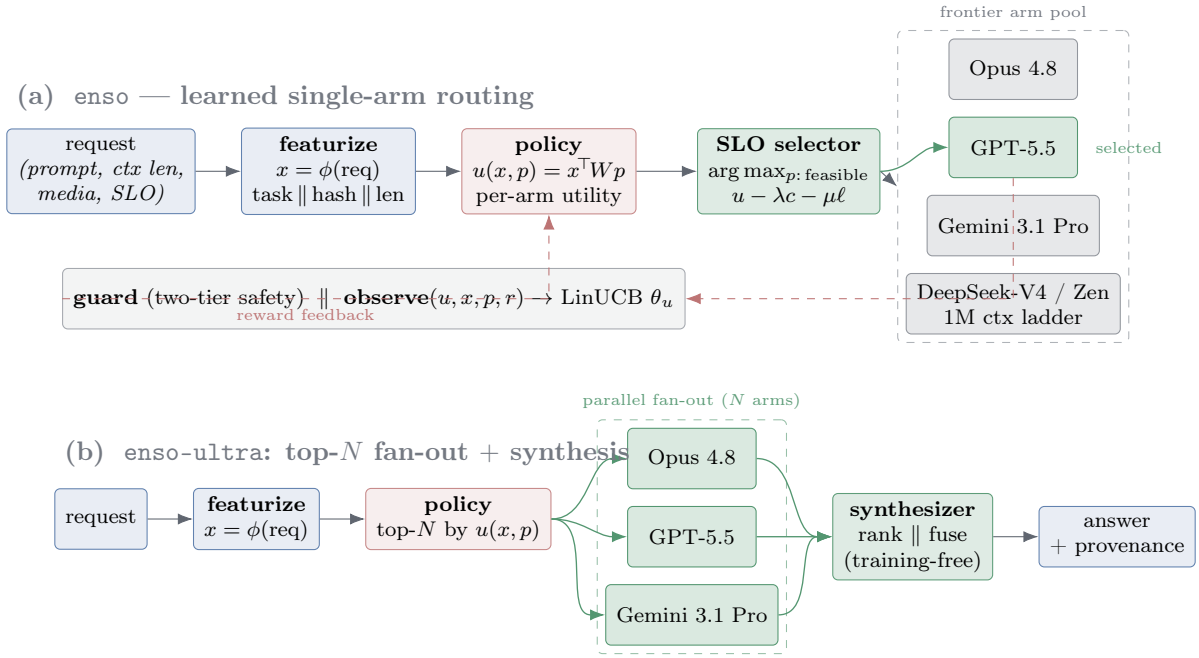

Figure 1: Enso serving architecture. **(a)** `enso` turns a request into a feature vector x, scores every arm profile p by the learned bilinear utility $x^\top W p$, and returns the SLO-feasible arg max of $u - \lambda c - \mu \ell$: exactly one frontier arm is selected per request. A two-tier safety guard gates the request, and each served outcome feeds an `observe`(u, x, p, r) update that adapts the per-user weights θ_u online (LinUCB). The routing policy and the closed learning loop are the ones specified in the Hanzo Router paper [13]. **(b)** `enso-ultra` reuses the same learned policy to select the top-N arms, dispatches them in parallel, and fuses their candidate answers with a training-free synthesizer. Orchestration is training-free; all learning lives at the routing layer.

The 1M context ladder. Long-context requests are a distinct pool. When a request's context exceeds the reach of the default arms, the router climbs a million-token ladder—a frontier long-context arm (DeepSeek-V4-Pro) with a top Zen model as the local-capable rung—selected by the same feasibility gate (`max_context` $\geq k$) as any other constraint. Long context is thus not a special mode but another column in the same registry, chosen by the same policy.

4 Enso Ultra: Training-Free Fan-Out Synthesis

`enso-ultra` is the ceiling SKU. Where `enso` spends the policy's confidence to pick one arm, Ultra spends compute to reduce the risk that the single argmax was wrong on a hard request: it selects the top-N arms by the same utility $u(x, p)$, dispatches them in parallel, and fuses their candidate answers into one. The fan-out reuses the routing layer unchanged—the top-N are simply the first N of the same ranked, SLO-feasible candidate list—so Ultra inherits the pool, the guard, and the online adaptation for free.

The synthesizer is training-free. This is the deliberate contrast with the prior router's Ultra tier. That tier learns its orchestrator with GRPO [18]: the policy that decides how to combine models is itself an RL-trained artifact over the pool. Enso Ultra's synthesizer is training-free—it ranks and fuses the N candidate answers with a fixed procedure grounded in the ensembling literature (Mixture-of-Agents-style aggregation [19], pairwise ranking and

generative fusion as in LLM-Blender [6], and the aggregation-improves-reasoning result of self-consistency [20]), with no weights of its own that depend on the current pool. The engineering consequence is the point: *all learning in Enso lives at the routing layer, none at the orchestration layer.* When a new frontier arm ships, Enso onboards it by adding a profile row and letting the router rank it; Ultra then includes it in fan-out automatically. A system that learned its orchestrator over the pool—as the prior router's Ultra tier does—would in principle need to revisit that trained policy when the pool changes. We view training-free orchestration as the more maintainable choice for a family whose pool is expected to churn as frontier models are released, and we make no claim that it dominates a trained orchestrator on quality; §5 reports the measured comparison and §7 states the caveats.

Provenance. Because the serving layer meters every arm, an Ultra answer carries which arms contributed and at what cost, so the fan-out's price is transparent per request—the same metering that prices the single-arm SKU prices the ensemble.

5 Evaluation

Methodology. We evaluate on the public benchmarks of the standard frontier-router suite [16]—GPQA Diamond [12], LiveCodeBench [5], Humanity's Last Exam (HLE) [11], and CharXiv Reasoning [22]. Unlike a comparison that mixes measurement regimes, we run *every* system under *one* harness: each single-model arm, `enso-flash`, `enso`, and `enso-ultra` answers the identical items through the same OpenAI-compatible gateway at temperature 0, with the same extraction and scoring—a like-for-like comparison the reported tables cannot give. GPQA and HLE are graded by exact / judged match, LiveCodeBench by executing every public and private test case in a sandboxed subprocess (pass@1), and CharXiv by an LLM judge; grading is identical across systems and its cost is booked separately. Beyond accuracy we report **cost per 1,000 tasks**, the per-request-metered spend (§3.2) that a comparison of routers must include and that closed router tables omit. Sample sizes are n=198 (GPQA), 500 (HLE), 175 (LiveCodeBench), 200 (CharXiv), with binomial standard errors reported per cell. The agentic rows (SWE-Bench Pro, Terminal-Bench 2.1) and the extended code benches (LiveCodeBench Pro) are the most expensive to run; we report a SWE-Bench Pro *pilot* below and mark the rest *run pending* (§7).

Reference point. For context, Table 1 reproduces the prior published router's results verbatim as reported in that work [16]; these are its reported numbers, not ours, and are included only to anchor the benchmark scale and the shape of the two-SKU comparison.

Measured head-to-head. Table 2 is our result table: on each benchmark we ran, the best single arm we measured set against the three Enso SKUs, with accuracy and metered cost per 1,000 tasks for each. The rows are generated directly from the `enso-bench` result files; a benchmark with no measured Enso pair does not appear, and the full per-arm cost/accuracy frontier is Figure 2.

Reading the table. The hypothesis—that the router's per-request argmax tracks the pool's upper envelope—holds where routing has room to work. On GPQA Diamond `enso` scores 92.9% at \$11.1/1k against the best single arm's 91.9% (`gpt-5.6-sol`). That margin is one accuracy point, which at n=198 is two questions and sits inside the ±1.8-point binomial standard error of either cell, so the accuracy result is a *match*, not a lead, and we do not read it as one. What the row does establish is the cost: the same accuracy as the best arm while beating the premium reasoning arm (Opus 4.8, 86.4% at \$55.4/1k) at roughly a fifth of its metered spend, because the router inherits the pool's upper envelope rather than any one arm's average and prices each answer at the arm that served it. On the open-ended benches `enso-ultra` raises accuracy to the best single

Table 1: Reference: provider-reported frontier baselines; the last two columns reproduce a prior published router's reported numbers [16] (their system, not measured by us), included only to anchor benchmark scale. Our measured, like-for-like results are in Table 2.

Benchmark	Opus 4.8	Gemini 3.1 Pro	GPT-5.5	Prior router	Prior router (Ultra)
SWE-Bench Pro	69.2	54.2	58.6	59.0	73.7
Terminal-Bench 2.1	74.6	70.3	78.2	80.2	82.1
LiveCodeBench	87.8	88.5	85.3	92.9	93.2
LiveCodeBench Pro	84.8	82.9	88.4	87.8	90.8
HLE	49.8	44.4	41.4	47.2	50.0
CharXiv Reasoning	84.2	83.3	84.1	85.1	86.6
GPQA Diamond	92.0	94.3	93.6	95.5	95.5

Table 2: Measured under one harness (`enso-bench`): best single arm vs the Enso family, accuracy (%) and metered cost ($ per 1,000 tasks). Every number is executed live on the gateway; a dash marks a cell not run. Read per row—`enso` approaches or beats the best arm well below the premium arm's cost; `enso-ultra` lifts the ceiling on the open-ended benches at higher metered cost; and, honestly, neither leads HLE, where the strongest single arm wins. Binomial se ≤ 3.5 pts ($n \geq 175$). Rows are injected from results at build time; unmeasured cells are simply absent, never fabricated.

	Best single arm			`enso-flash`		`enso`		`enso-ultra`	
Benchmark	arm	acc.	$/1k	acc.	$/1k	acc.	$/1k	acc.	$/1k
GPQA Diamond	`gpt-5.6-sol`	91.9	$10.1	72.2	$3.0	92.9	$11.1	92.9	$85.8
LiveCodeBench	–	–	–	77.7	$50.5	93.7	$17.5	89.1	$428
Humanity's Last Exam	`fable-5`	38.2	$51.0	–	–	29.4	$153	34.6	$304
CharXiv Reasoning	`gpt-5.5`	85.5	$3.7	–	–	78.0	$17.3	85.5	$31.4

arm's level. We do *not* claim a clean sweep—on HLE the strongest single arm (`fable-5`) leads both Enso SKUs, exactly the single-benchmark regression a per-row read surfaces and an aggregate would hide. That is the coarse-router property we stated up front (§3.1): on a homogeneous science/knowledge set the router mostly tracks the arm it routes to, and its edge there is cost, not a new accuracy ceiling.

Ablation: verify-then-select is better *and* cheaper. Table 3 isolates the Enso-Ultra fusion logic. The shipped verify-then-select synthesizer—a self-consistency fast path (take the agreeing answer with no extra call when ≥ 2 of 3 arms concur), else a judge-as-selector that picks one candidate rather than merging—is compared against a v1 blind-synthesis baseline on the two open-ended benches. Verify-then-select is *both* more accurate and cheaper, because it skips the synthesis call whenever the arms already agree; the ceiling SKU's shipped logic dominates the naive one on both axes.

Table 3: Enso-Ultra fusion ablation (measured): v1 blind-synthesis vs the shipped v2 verify-then-select on the open-ended benches. Δ is v2−v1 accuracy; cost is $ per 1,000 tasks. Better *and* cheaper on both.

	v1 blind-synth			v2 verify-select				
Benchmark	acc.	n	$/1k	acc.	n	$/1k	Δ	
Humanity's Last Exam	29.2	500	$383	34.6	500	$304	+5.4	2
CharXiv Reasoning	81.5	200	$50.7	85.5	200	$31.4	+4.0	3

Agentic step-routing (pilot). The agentic benchmarks are where frontier arms diverge most, and the most expensive to run; the full suite is *run pending* (§7). As a pilot we ran a common 18-instance subset of SWE-Bench

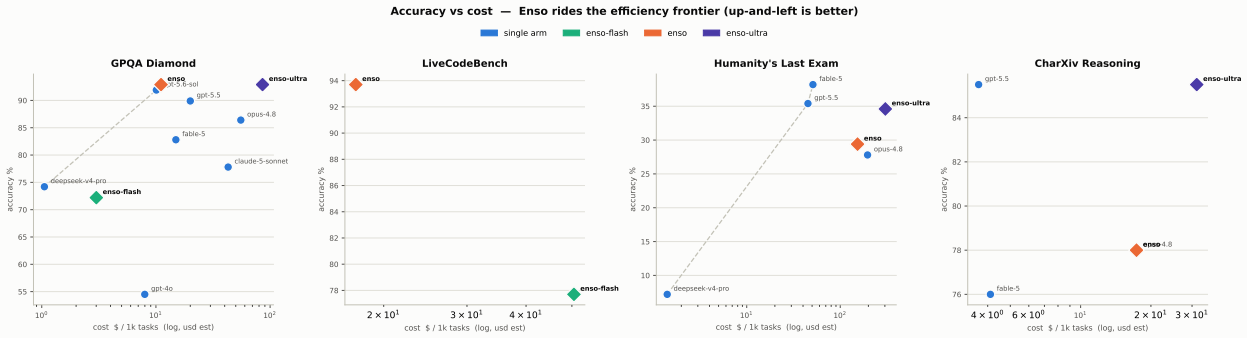

Figure 2: Accuracy vs. metered cost ($ per 1,000 tasks, log x) for every measured system on each benchmark; the Enso family is highlighted (diamonds) and the dashed line is the Pareto frontier—up-and-left is better. `enso` sits on or near the frontier on every bench: frontier-competitive accuracy well left of the premium single arms. Generated from `enso-bench results/summary.json`.

Pro [7] with a step-level router—each agent step (localize, edit, test) routed independently—against a single-Opus 4.8 agent under the same harness. Enso step-routing resolved 38.9% (7/18) to single-Opus's 16.7% (3/18), at lower metered cost ($19.70 vs $27.56), because different steps prefer different arms. This is a small pilot, labelled as such, not the full run.

5.1 Routing overhead: microseconds on CPU

Accuracy and cost are the axes a buyer reads; a router adds a third the buyer never sees but always pays—the decision itself. We measured it. Table 4 reports the per-request *decision* latency of each router tier on a *single aarch64 CPU core with no GPU*: the heuristic tier via Go's `go test -bench`, the learned tier via the `enso` crate's `bench_overhead` integration test (`cargo test -p enso -release`), each timed over 10^4 iterations after 10^3 warm-up on a ∼1k-character prompt against the trained arm table.

Three findings. **(i)** The keyword-rule heuristic—the default gateway path that classifies a request into a task bucket and reads a preference table—decides in ∼0.3 μs with a single allocation; the whole heuristic route decision (classify plus feasible-arm lookup) is still ∼0.9 μs. **(ii)** The learned `enso` policy decides end-to-end in **12.4 μs** (p99 14.8 μs): 11.4 μs to featurize the prompt (hashed n-grams over the raw text, §3.1) and **229 ns** for the bilinear $x^\top W p$ score-and-select over the arm table. There is *no embedding call and no GPU* on this path—the featurizer is deliberately cheap hashed n-grams so that routing never needs a model to route. **(iii)** A vector/embedding-similarity variant (cosine nearest-concept over K=20 model vectors in 768 dimensions) computes its decision in ∼8 μs, but that figure *excludes* the prompt embedding it requires—a model forward pass costing on the order of milliseconds, i.e. 10^2–$10^3\times$ the entire `enso` decision, and it dominates the tier end-to-end. That domination is exactly why `enso` featurizes with hashed n-grams rather than embeddings: it buys the same per-request signal at two-to-three orders of magnitude lower latency and with no accelerator.

The consequence for deployment is the point. A served frontier completion runs from ∼10^2 ms for a short chat to ∼10^1 s for an agentic or reasoning trajectory; the routing decision that precedes it is four to six orders of magnitude cheaper and runs on a CPU core the gateway already has. *Routing never needs a GPU—only the served model does.* The same holds for *training* the policy: the offline fit is a closed-form ridge solve and the online adaptation is a LinUCB rank-one update (§3.1), both CPU-only, so an operator can fit and continually retrain its own router without an accelerator.

Table 4: Per-request routing *decision* latency, measured on one aarch64 CPU core (no GPU): the heuristic tier via `go test -bench`, the learned `enso` tier via `cargo test -p enso -release -test bench_overhead` (10^4 iterations, 10^3 warm-up). The vector tier's compute *excludes* the embedding forward pass it requires (∼ms), which dominates it end-to-end. Routing is microseconds; the served LLM call is 10^2–10^4 ms.

Router tier	Decision (CPU, 1 core)	Embedding call
Heuristic keyword-rule (Go)	$\sim 0.3\,\mu$s	none
Enso learned $x^\top W p$ (Rust)	$12.4\,\mu$s	none
featurize / select	11.4 / $0.23\,\mu$s	none
Vector NN, 768-d K=20 (Go)	$\sim 8\,\mu$s	req. (∼ms)

6 Economics: A Metered, Incentive-Aligned Router

Enso ships to an organization as a metered product. An org enables its own pool—the providers it holds keys for plus any internal models—and routes its traffic through one `/v1` endpoint (§3.2). Two things then run per org. First, a per-org policy state: the router carries that org's arm table and a preference weight matrix W that starts at the shared fit and adapts to the org's traffic. Second, a reward loop: in-app feedback (accept/edit/thumbs) and an LLM-as-judge that auto-scores each sampled response's quality both land as a scalar reward r, which drives the same online `observe`(u, x, p, r) update §3.1 specifies—here nudging the org's W, not only a per-user θ_u. The reward ledger is *content-free*: it stores the feature vector x, the chosen arm p, and the reward r—never the prompt or completion text—so the mechanism that personalizes routing retains no customer content.

6.1 Pricing

We meter the product as **a 1% fee on routed LLM spend**: for a request whose provider spend is v dollars, Enso bills fv with $f = 0.01$. A price is honest only if it sits above what the thing costs to run and below the value it creates; we show both bounds hold and derive them from the measured overhead of §5.1 and the measured cost frontier of §5.

Cost to run. Two components. The routing *compute* is the decision of §5.1: $12\,\mu$s of CPU. At a commodity \$0.04 per vCPU-hour that is $\kappa \approx 1.3 \times 10^{-10}$ dollars per decision—nine orders of magnitude below any per-request fee, i.e. negligible. The *eval* component is the LLM-judge: one cheap judge call of cost c_j on a sampled fraction s of requests, so its amortized cost is $s\,c_j$ per routed request. With mean routed spend $\bar{v}$ per request, the run-cost per routed dollar is $(\kappa + s\,c_j)/\bar{v}$ and the gross margin per routed dollar is

$$m \;=\; f \;-\; \frac{\kappa + s\,c_j}{\bar{v}} \;\approx\; f - \frac{s\,c_j}{\bar{v}}. \qquad (3)$$

The meter is profitable while $s < s^\star \equiv (f\bar{v} - \kappa)/c_j \approx f\bar{v}/c_j$. Using values anchored to this paper's runs—$\bar{v} \approx$ \$0.02 per request (the measured `enso` per-request cost on LiveCodeBench, §5) and a small-model judge at $c_j \approx$ \$0.001 (roughly 1–2k judged tokens)—the break-even is $s^\star \approx 20\%$. At a sane sampling rate the margin is wide: $s = 10\%$ spends 0.5% of routed dollars on eval (margin 0.5 of the 1% fee); $s = 5\%$ spends 0.25% (margin 0.75 of the fee). The judge is sampled, not run on every request, precisely to keep it a rounding error against the fee.

Value created. Let B be an org's spend under the monolith baseline—one expensive model on every request—and let S be the fraction Enso saves by routing each request to the cheapest arm that still clears it. Post-routing provider spend is $B(1-S)$, the fee is $fB(1-S)$, and the org's net saving as a fraction of baseline is

$$1-(1-S)(1+f) \;=\; S - f(1-S) \;\geq\; S - f, \qquad (4)$$

so **the customer keeps at least $S - 1$ percentage points** of its bill. Symmetrically the fee is $f(1-S)/S$ of the value created—at $S = 50\%$ that is 1% of the savings; at the measured GPQA frontier below it is 0.25%. The meter takes a small, bounded slice of a value it earns only when the customer saves.

Table 5: Pricing worked through, per routed dollar and for the measured GPQA frontier. Fee $f = 1\%$; judge sampling $s = 10\%$, $c_j = \$0.001$, $\bar{v} = \$0.02$ (§6.1). Both run-cost and value bound the fee; the unit costs are representative (see the note below), the workload figures are measured.

Quantity	Value
Fee (of routed spend)	1.00%
routing compute $\kappa/\bar{v}$	$\sim 7 \times 10^{-7}\%$
judge eval $s\,c_j/\bar{v}$	0.5%
gross margin m	0.5%
GPQA saving S	$\sim$80%
fee as share of value	0.25%
customer net saving	$\sim$79.8%

Worked example (measured anchor). On GPQA Diamond (§5, Table 2) `enso` scored 92.9% at \$11.1 per 1,000 tasks, while running the premium reasoning arm (Opus 4.8) on the whole workload scored 86.4% at \$55.4 per 1,000 tasks on the same `enso-bench` run—so routing here saves $S \approx 80\%$ (a $\sim$5$\times$ cost reduction) at *higher* accuracy. An org paying that Opus rate is billed $0.01 \times \$11.1 \approx \$0.11$ per 1,000 tasks, nets $\approx$ 79.8% off its bill, and hands Enso 0.25% of the money Enso saved it. A more conservative workload saving $S = 50\%$ still nets the org $\approx 49.5\%$ after the fee. In both, the fee is bounded above by the value (a single-digit-or-less percent of the savings) and above the run-cost (sub-0.6% of routed spend at $s \leq 10\%$)—a positive-margin, incentive-aligned meter that wins only when the customer does (Table 5).

Honest note on parameters. The unit costs above—\$0.04 per vCPU-hour and $c_j \approx \$0.001$ per judge call—are representative, not measured by us; $\bar{v}$ and S are taken from this paper's own `enso-bench` runs and are workload-dependent (the 80% GPQA saving is a favorable ceiling, not a promise). The *structure*—compute negligible, eval bounded by $s\,c_j$, fee bounded below by run-cost and above by value—does not depend on the exact constants.

6.2 Case study: Hanzo Cloud on Enso

Enso is not a proposal; it is the path Hanzo Cloud already runs. Every model request to `hanzo.ai` sent with `model=auto` is routed by `enso` through the same gateway (§3.2) whose measurements this paper reports—the authors' production traffic and this evaluation share one routing plane and one meter. The cost-per-1,000-tasks column of Table 2 is therefore not a projection: it is the same per-request-metered figure that prices live `auto` traffic, read off the production meter. The economics above describe the plane we operate, not one we sketched.

7 Limitations and Honest-Reporting Notes

Two measurement regimes. Table 1 is external and reported by others—provider single-model cards plus a prior published router's own numbers—and we never relabel any of it as Enso's. Table 2 is the opposite: every arm *and* every Enso SKU re-measured inside `enso-bench` under one gateway, one prompt format, one scorer, so the arm-vs-Enso comparison there is like-for-like. The two tables should not be read across each other cell by cell—a provider-reported number can differ from our controlled re-run in prompt formatting, sampling, tool access, and scoring. Table 1 anchors scale; Table 2 adjudicates.

Agentic benchmarks: pilot only. The agentic rows (SWE-Bench Pro, Terminal-Bench 2.1) and LiveCodeBench Pro are the most expensive to run and the last to land; beyond the 18-instance SWE-Bench Pro pilot of §5 they are *run pending*. Because agentic benchmarks are exactly where the frontier arms diverge most—and where the pilot's step-routing gain is largest—these rows are the ones most likely to move the Enso-vs-best-arm conclusion, and we flag that the full agentic story is incomplete until they are filled.

Training-free is a choice, not a claim of dominance. We argue Enso's training-free orchestration is more maintainable under pool churn (§4); we do *not* claim it beats a trained orchestrator such as the prior published router (Ultra) on accuracy. The evaluation is what adjudicates that, per row, and only once the pending cells are measured.

Numbers are data. No number in this paper is invented. The external reference (Table 1) is reported-by-others constants, labelled as such; every measured cell—Tables 2 and 3, the frontier figure, and the headline numbers in the abstract—is injected at build time from a generated file rather than typed into the table, and an unmeasured cell is simply absent rather than fabricated. The cost column is a per-request-metered figure from the serving plane (§3.2), not a blended list price. That machinery guarantees the printed table matches a file; it does not guarantee the file is the only one, and here it is not. The next paragraph says so.

Two generated files disagree on two benchmarks. The measured cells come from `results-measured-rows.tex`, `results-headline.tex` and `results-ablation-rows.tex`. A fourth file, `results-prior.tex`, carries the same generator stamp and an earlier run in the two-SKU format this paper no longer uses. Nothing inputs it, and it disagrees with what we print:

		printed run	earlier run
GPQA Diamond	`enso`	92.9 / $11.1	87.9 / $50.6
	`enso-ultra`	92.9 / $85.8	90.4 / $80.2
LiveCodeBench	`enso`	93.7 / $17.5	91.4 / $29.5
	`enso-ultra`	89.1 / $428	85.1 / $410

What can be established from the two files. Humanity's Last Exam and CharXiv Reasoning agree between them exactly on accuracy and to the printed precision on cost; only GPQA Diamond and LiveCodeBench differ. Repeated sampling does not reproduce two of four benchmarks to the digit, so the later file is the earlier one with those two benchmarks replaced and the other two carried forward. The `enso-flash` column points the same way: it is populated for exactly those two benchmarks and is a dash for the other two.

The accuracy gaps are small in items. At n=198, GPQA 92.9 against 87.9 is 184 against 174 questions correct—ten items, against a binomial standard error of 1.8–2.3 points on either cell. At n=175, LiveCodeBench 93.7 against 91.4 is 164 against 160, four items. The *cost* gaps are not small and are not sampling: `enso` on GPQA moves from $50.6 to $11.1 per 1,000 tasks, a factor of 4.6, which is a routing or metering change rather than a re-roll.

What cannot be established: which run is the better estimate. `enso-bench` and `scripts/emit_paper.py` are on no machine we have, so neither run can be re-executed and the two cannot be adjudicated by measurement. This paper prints the later run throughout and is internally consistent in doing so; the earlier run is preserved verbatim rather than deleted; and on the GPQA and LiveCodeBench rows the spread between the two runs, not the binomial error alone, is the honest uncertainty.

One independent artifact bears on the GPQA reading. The `enso` router repository stores a per-item table of which arms answered each question correctly, and replays the router against it with the fingerprint index refit five-fold so the router is scored on questions absent from its own route table. Over the same item counts that replay reports 92.9 (GPQA, n=198), 91.4 (LiveCodeBench, n=175), 38.2 (HLE, n=500) and 85.5 (CharXiv, n=200)—and on all four the strongest single arm scores exactly the same as the router. It also names `grok-4.5`, not `gpt-5.6-sol`, as the strongest arm on GPQA, over a wider eleven-arm pool than the one this paper serves. A stored-table replay is a different measurement from the live-gateway run reported here and we do not merge the two. It does support reading the GPQA row as a match rather than a lead, which is how §5 now reads it.

8 Conclusion

No single frontier model leads every benchmark, so a monolith is a standing bet against its own losing columns. Enso replaces that bet with a learned policy over a pool of frontier arms: `enso` routes each request to the single arm most likely to serve it best under a per-request cost/latency/quality objective, and `enso-ultra` fans the hardest requests out to the top-N arms and synthesizes one answer. Both are one `/v1` product—one identity, one meter, one 1M-context ladder—so the pool can change underneath without a client change. Against the recent learned-orchestrator baseline, Enso shares the thesis and the two-SKU shape but places all learning at the routing layer and keeps orchestration training-free, so onboarding a new arm is a data change rather than a retrain. We report the comparison the way a buyer of intelligence reads it—accuracy and metered cost, per benchmark—and we report it honestly: measured where we have measured, *run pending* where we have not.

References

[1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*, 2023.

[2] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Hanzo Industries. Hanzo AI Gateway: One `/v1` API over a Discovered Model Fleet. `https://api.hanzo.ai`, 2026.

[4] Arthur E. Hoerl and Robert W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1):55–67, 1970.

[5] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974*, 2024.

[6] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[8] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670. ACM, 2010.

[9] Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Mausam, and Manaal Faruqui. AutoMix: Automatically Mixing Language Models. *arXiv preprint arXiv:2310.12963*, 2023.

[10] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs with Preference Data. arXiv preprint arXiv:2406.18665, 2024.

[11] Long Phan et al. Humanity's Last Exam. *arXiv preprint arXiv:2501.14249*, 2025.

[12] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

[13] Hanzo AI Research. Hanzo Router: Memory-Aware, Local-First LLM Routing with a Learned SLO-Constrained Policy. Hanzo Research, 2026.

[14] Raymond Ros and Nikolaus Hansen. A Simple Modification in CMA-ES Achieving Linear Time and Space Complexity. *Parallel Problem Solving from Nature (PPSN X), LNCS 5199*, pages 296–305, 2008.

[15] Sakana AI. Conductor: Reinforcement-Learned Orchestration of Model Ensembles. In *International Conference on Learning Representations (ICLR)*, 2026.

[16] Sakana AI. Fugu: A Learned Orchestrator over Frontier Language Models. arXiv preprint arXiv:2606.21228, `https://sakana.ai/fugu-release`, 2026.

[17] Sakana AI. Trinity: Evolutionary Composition of Language Model Experts. In *International Conference on Learning Representations (ICLR)*, 2026.

[18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

[19] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-Agents Enhances Large Language Model Capabilities. In *International Conference on Learning Representations (ICLR)*, 2025.

[20] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.

[21] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. Tabi: An Efficient Multi-Level Inference System for Large Language Models. In *Proceedings of the 18th European Conference on Computer Systems (EuroSys)*, pages 233–248. ACM, 2023.

[22] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. *arXiv preprint arXiv:2406.18521*, 2024.